Deep Reinforcement Learning with Value Function and Policy Approximation.
Actor-Critic Methods for Continuous Control: PPO and SAC

José Ridruejo Tuñón, Joaquín Mir Macías & Francisco Javier Ríos
Deep Reinforcement Learning

ABSTRACT
This work studies deep reinforcement learning for continuous control from visual observations using MuJoCo environments accessed through Gymnasium. The focus is on Actor-Critic algorithms with explicit policy approximation: Proximal Policy Optimization (PPO), an on-policy method, and Soft Actor-Critic (SAC), an off-policy maximum-entropy method. We train both agents on the Hopper-v5 locomotion task from pixel observations with a discretized action space, analyzing the impact of reward design, visual state representation, and hyperparameter choices on learning dynamics. We experimentally compare PPO and SAC in terms of sample efficiency, training stability and final performance. PPO, trained for approximately ten days ($\sim$251,000 rollout episodes), achieved a best evaluation reward of 3783, while SAC, trained for only $\sim$16 hours ($\sim$8,000 episodes), already reached 2510, which shows superior sample efficiency. The results highlight the practical trade-offs between on-policy and off-policy Actor-Critic methods in visually rich continuous control, as well as the role of entropy regularization, action discretization, and reward shaping in shaping the learning process.

Key words: Deep Reinforcement Learning, Actor-Critic, Proximal Policy Optimization, Soft Actor-Critic, MuJoCo, Gymnasium, Continuous Control, Visual Control, Entropy Regularization

Repository: `https://github.com/Javirios03/P3_PPO_SAC`

1 INTRODUCTION

In this practice we address continuous control problems from the perspective of Actor-Critic methods with explicit policy parameterization. Unlike in the previous assignment, where the policy was implicit and derived from an approximated action-value function $Q(s, a)$, here the agent learns a stochastic policy $\pi_\theta(a \mid s)$ together with a value function used as critic. The state of the environment is provided exclusively through RGB images, so the agent must simultaneously learn a visual representation and a control policy on top of it.

The physical simulation is handled by MuJoCo, a physics engine widely used in research on locomotion and manipulation, while the tasks are defined through Gymnasium. In this project we implement and adapt two representative Actor-Critic algorithms: Proximal Policy Optimization (PPO), an on-policy method based on a clipped surrogate objective and Generalized Advantage Estimation (GAE), and Soft Actor-Critic (SAC), an off-policy method based on the maximum entropy RL framework with replay buffer, double critic, and target networks.

The goals of the practice are: (i) to understand theoretically and empirically how PPO and SAC work in continuous control with visual observations, (ii) to configure and train agents in locomotion environments, (iii) to compare on-policy vs. off-policy behaviour in terms of sample efficiency, stability and final performance, and (iv) to extend the analysis to a more demanding setting where the agent must both move forward and avoid obstacles. The report is structured in three parts: PPO locomotion, SAC locomotion and comparative analysis of the two, followed by general conclusions.

2 PPO FOR LOCOMOTION

2.1 Methodology

2.1.1 Environment and observation space

We use the `Hopper-v5` environment from Gymnasium with MuJoCo physics. The Hopper is a planar one-legged robot with three actuated joints (hip, knee, ankle) whose goal is to reach the end of the track (hopping by using its only leg, by means of twisting the joints) without falling in the process. The observation consists of an RGB render of the scene, resized to 84×84 grayscale pixels and stacked over the last 4 frames, giving an input tensor of shape $(4, 84, 84)$. Frame stacking provides the policy with an implicit sense of velocity, which is essential since a single frame cannot disambiguate direction of motion.

2.1.2 Action space discretization

Since PPO as implemented here uses a Categorical distribution, and Hopper has a continuous action space $\mathcal{A} \subset [-1, 1]^3$, we discretize it using a product grid. For each of the 3 joints we linearly space $B = 3$ values in $[-1, 1]$ (i.e., $\{-1, 0, +1\}$) and form all $B^3 = 27$ combinations. The action at each step is therefore one of 27 joint-torque vectors, covering not only single-joint but also coordinated multi-joint movements — which are indispensable for effective hopping. This is a critical design choice: an early implementation that generated only one-hot actions (one joint at a time, 7 total) created a degenerate action space where coordinated hopping was physically impossible.

2.1.3 Network architecture

Both actor and critic share a convolutional encoder with architecture $(4 \to 32, k{=}8, s{=}4) \to (32 \to 64, k{=}4, s{=}2) \to (64 \to 64, k{=}3, s{=}1) \to$ Flatten, each layer followed by ReLU. The flattened representation (1024-dimensional for 84×84 input) is fed into separate actor and critic heads, each a two-layer MLP with 512 hidden units. The actor head outputs logits over the 27 actions, from which a Categorical distribution is sampled. The critic head outputs a scalar state value $V(s)$. All weights are initialized orthogonally with gain $\sqrt{2}$; the final actor layer uses gain 0.01 to produce near-uniform initial policies.

2.1.4 PPO algorithm and hyperparameters

PPO optimizes the clipped surrogate objective:

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min\Big(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1{-}\epsilon, 1{+}\epsilon)\hat{A}_t \Big) \right], \tag{1}$$

where $r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{\text{old}}}(a_t|s_t)$ is the importance ratio and $\hat{A}_t$ is the Generalized Advantage Estimate (GAE). The full loss adds a value function term and an entropy bonus:

$$\mathcal{L}(\theta) = \mathcal{L}^{\text{CLIP}} - c_1\mathcal{L}^{\text{VF}} + c_2\mathcal{H}[\pi_\theta]. \tag{2}$$

Advantages are computed via GAE with $\lambda = 0.95$. The key hyperparameters used are summarized in Table 1.

Table 1: PPO hyperparameters for Hopper-v5 with pixel observations.

Hyperparameter	**Value**
Rollout steps per update	2048
Update epochs per rollout	10
Batch size	1024
Learning rate	10^{-4}
Discount γ	0.99
GAE λ	0.95
Clip coefficient ϵ	0.2
Critic coefficient c_1	0.2
Entropy coefficient c_2	0.001
Max gradient norm	0.5
Reward: forward weight	0.5
Reward: healthy reward	1.25
Reward: control cost weight	0.1
Max episode steps	20000

2.2 Results and Discussion

2.2.1 Theoretical Analysis

Before analysing the empirical results, we first consider the key theoretical aspects of PPO that we expect to influence its performance on the locomotion task.

The first aspect to consider is the nature of the policy update in PPO Schulman et al. (2017). The clipped surrogate objective is designed to prevent large updates that could destabilize training, which marks its main improvement with respect to earlier on-policy methods like TRPO. However, this also means that PPO may be more conservative in its updates, potentially leading to slower learning in environments where significant changes in the policy are needed to make progress.

PPO is an on-policy algorithm. The data used for training the policy is generated by the current policy, which simplifies the learning process and tends to produce more stable updates because the data distribution is always aligned with the current policy. However, this implies much lower sample efficiency compared to off-policy methods: once a rollout is collected and used for K epochs of optimization, the data is discarded, with no ability to reuse transitions from earlier in training.

Due to this, we expect PPO to require a larger number of environment interactions to achieve good performance. On the other hand, it should be more robust to hyperparameter choices and less sensitive to the quality of the value function approximation, since the policy updates are more constrained by the clipping mechanism. We also expect training with visual observations to be particularly challenging: the convolutional encoder must learn a compact representation of the environment purely from raw pixels, without access to proprioceptive state variables. This increases the number of samples needed to form a useful critic.

A secondary concern for locomotion is the reward structure. Because Hopper naturally has a large healthy-reward component (simply remaining upright and alive), a poorly tuned reward can cause the policy to converge to a standing-still local optimum. We expect PPO to be somewhat more tolerant of this, as the entropy bonus prevents premature policy collapse.

2.2.2 Empirical Results

The full training of the PPO agent in the Hopper environment with RGB observations ran for approximately ten days (from March 28 to April 7), covering $\sim$251,000 rollout episodes and around 3×10^6 total environment steps. Despite this substantial computational effort, the training reward peaked well before the millionth step, suggesting that the most significant learning occurred in the first 3–4 days. The smoothed mean reward over the last 100 episodes reached a maximum of 2720, and the best evaluation episode (single run, greedy policy) achieved a reward of 3783. Figure 1 shows the learning curves for reward, actor loss, critic loss, and entropy.

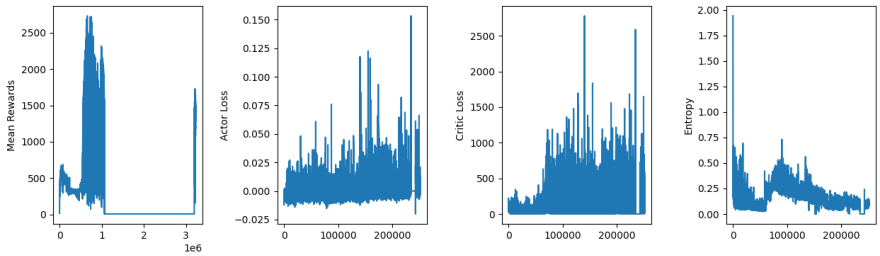
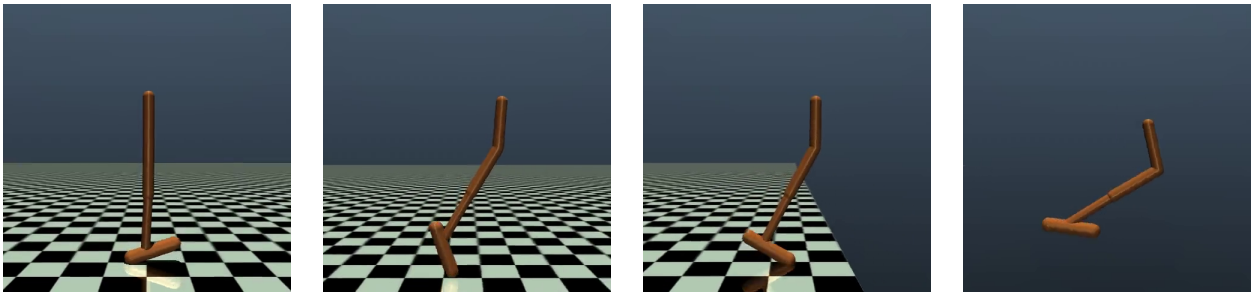


Figure 1: Learning curves for PPO training on Hopper-v5 with pixel observations. From top to bottom: smoothed episode reward (mean over 100 episodes), actor loss, critic loss, and policy entropy. The reward curve shows a clear upward trend followed by a plateau and eventual regression, consistent with on-policy collapse after the policy reaches a local optimum.

Figure 2 shows four representative frames extracted from the best evaluation episode. The agent clearly learns to hop forward: frame 1 shows the initial upright stance; frame 2 shows a pronounced mid-air hop with the thigh and knee fully engaged; frame 3 shows the landing phase with the foot flat on the ground; and frame 4 shows the end of the episode, where the hopper has traveled far enough off-camera that the camera no longer tracks it — indicating a fall after sustained forward locomotion.



(a) Upright stance (b) Mid-air hop (c) Landing phase (d) Fall (episode end)

Figure 2: Four representative frames from the best PPO evaluation episode (reward = 3783). The agent learns a clear hopping gait: it pushes off the ground with coordinated joint torques, achieves substantial forward displacement, and ultimately falls after a long run.

A notable feature of the training is that the mean reward curve collapsed after reaching the peak. This behavior after a performance peak is a well-known failure mode of on-policy methods: once the policy starts changing significantly from the data-generating policy, the clipping constraint is effectively inactive, and the optimization can move the policy into an unstable region. This is consistent with the theoretical expectation that on-policy methods, while more stable in early training, can suffer from catastrophic forgetting in the long run when no entropy regularization is strong enough to maintain behavioral diversity.

Some key observations from the training process:

- **Sample efficiency**: As expected, PPO required a very large number of episodes to reach peak performance — a known limitation of on-policy methods. Early training showed high variance in actor and critic losses.
- **Hyperparameter sensitivity**: PPO was relatively robust to changes in the learning rate and clipping parameter. The reward weights had a more significant impact, particularly `healthy_reward`, which was kept at 1.25 to prevent the standing-still local optimum.
- **Stability**: Training was generally stable until the peak, after which it regressed. This late-stage instability is consistent with on-policy policy drift.
- **Behavioral quality**: The trained agent shows a clear, energetic hopping gait. The full-body coordination visible in the mid-hop frame (Figure 2b) confirms that the product-grid action discretization was essential: the agent actively uses simultaneous multi-joint torques.

2.3 Conclusions

PPO successfully learns a hopping policy for Hopper-v5 from pixel observations, reaching a best evaluation reward of 3783 after approximately ten days of training. The result confirms the theoretical prediction that on-policy methods are sample-inefficient but robust: despite requiring ∼251,000 episodes, the training was largely stable and did not require careful tuning beyond the reward weights. The main practical limitation observed is the post-peak regression, which suggests that PPO benefits from early stopping or adaptive learning-rate scheduling in long training runs. The action discretization scheme (27-action product grid) proved critical — the original one-hot discretization (7 actions) made coordinated hopping physically impossible, and the agent converged to a standing-still local optimum regardless of reward shaping.

3 SAC FOR LOCOMOTION

3.1 Methodology

3.1.1 SAC algorithm overview

Soft Actor-Critic (Haarnoja et al., 2018) is an off-policy actor-critic algorithm based on the maximum entropy RL framework. The agent maximizes the following objective:

$$J(\pi) = \sum_t \mathbb{E}_{(s_t,a_t)\sim\rho_\pi} \left[r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot \mid s_t)) \right], \tag{3}$$

where $\alpha > 0$ is the temperature parameter controlling the trade-off between reward maximization and entropy maximization. The entropy bonus encourages exploration and prevents the policy from collapsing to a deterministic solution prematurely.

The critic is parameterized as a Twin Q-network (Van Hasselt et al., 2015): two independent Q-networks Q_{ϕ_1}, Q_{ϕ_2} are trained, and the minimum of the two is used for policy improvement, which mitigates the overestimation bias inherent in Q-learning. Soft target networks $Q_{\bar{\phi}_i}$ are maintained via exponential moving average:

$$\bar{\phi}_i \leftarrow (1-\tau)\bar{\phi}_i + \tau\phi_i. \tag{4}$$

The temperature α is adjusted automatically by minimizing:

$$\mathcal{L}(\alpha) = \mathbb{E}_{a\sim\pi} \left[-\alpha \log \pi(a \mid s) - \alpha\bar{\mathcal{H}} \right], \tag{5}$$

where $\bar{\mathcal{H}}$ is a target entropy. For a discrete action space of size n, we set $\bar{\mathcal{H}} = -\beta_H \log(1/n)$ with $\beta_H = 0.45$ (this controls how close to uniform the policy is encouraged to be).

3.1.2 Architecture and adaptation for discrete actions

Since Hopper's action space is also discretized (same 27-action product grid as in PPO), we use the discrete SAC formulation (Christodoulou, 2019): the actor outputs a full probability vector over all 27 actions, and the critic outputs a Q-value for each action, avoiding the need to sample actions from the critic's perspective. This makes both the Bellman backup and the actor loss exact in expectation:

$$\mathcal{L}(\phi) = \mathbb{E}\left[\left(Q_\phi(s,a) - r - \gamma\, \mathbb{E}_{a'\sim\pi}[\min_i Q_{\bar{\phi}_i}(s',a') - \alpha \log \pi(a'|s')] \right)^2 \right], \tag{6}$$

$$\mathcal{L}(\theta) = \mathbb{E}_s \left[\sum_a \pi_\theta(a|s) \left(\alpha \log \pi_\theta(a|s) - \min_i Q_{\phi_i}(s,a) \right) \right]. \tag{7}$$

The actor and each of the two critics have independent convolutional encoders (same architecture as PPO: three Conv2D layers followed by a 512-unit MLP head). Using separate encoders for actor and critics prevents gradient interference and was found empirically to improve stability in pixel-based SAC.

3.1.3 Replay buffer and training procedure

SAC stores transitions (s, a, r, s', d) in a replay buffer of size 200,000. Training begins after 5,000 steps of random exploration. Thereafter, one gradient update is performed per environment step (`train_freq`$= 1$), sampling a mini-batch of 256 transitions uniformly from the buffer. This $1:1$ ratio of environment steps to gradient steps is the standard SAC regime and is what primarily distinguishes it from PPO in terms of data reuse. The hyperparameters are summarized in Table 2.

Table 2: SAC hyperparameters for Hopper-v5 with pixel observations.

Hyperparameter	**Value**
Replay buffer size	200,000
Batch size	256
Learning rate (actor & critic)	3×10^{-4}
Learning rate (α)	10^{-4}
Discount γ	0.99
Soft update τ	0.005
Initial α	0.2
Target entropy coef. β_H	0.45
Learning starts (random steps)	5,000
Train frequency	1 step/update
Max gradient norm	10.0
Reward: forward weight	0.7
Reward: healthy reward	0.6
Reward: control cost weight	0.05
Max episode steps	20,000

3.2 Results and Discussion

3.2.1 Theoretical Analysis

As we did previously for PPO, we first study the key theoretical aspects of SAC that we expect to influence its performance on the Hopper locomotion task.

The fundamental advantage of SAC is its off-policy nature: every transition stored in the replay buffer can be reused for multiple gradient updates at different points in training. This makes SAC dramatically more sample-efficient in terms of the number of environment interactions needed to reach a given performance level. However, this efficiency comes with its own challenges. Off-policy learning introduces a distribution shift between the data in the buffer (collected by older policies) and the current policy, which can destabilize the critic update when the function approximation is poor. The use of target networks and the twin-critic architecture are specifically designed to mitigate this instability.

The maximum entropy framework provides a second key advantage for exploration. By explicitly rewarding entropy, SAC maintains a diverse distribution over actions throughout training, avoiding the premature convergence to local optima that on-policy methods must handle through explicit entropy regularization (the `ent_coef` in PPO). The automatic tuning of α means that the exploration pressure is self-regulating: if the policy becomes too deterministic, α increases to push it back toward diversity.

For the pixel-based setting, SAC faces an additional challenge that PPO does not encounter too much: the critic must learn to map high-dimensional pixel inputs to Q-values for each action, and early in training, with a random or near-random encoder, the Q-value estimates are highly noisy. This can cause the auto-α mechanism to initially push the temperature very high (since low-quality Q-values make all actions look equally bad, leading to a uniform policy, which satisfies the entropy target easily). As the encoder improves, the Q-values become more meaningful, α decreases, and the policy sharpens. This "encoder cold-start" problem is a well-known difficulty for pixel-based SAC that does not afflict state-based SAC.

Given the much shorter training time allocated to SAC in this work, we expect it to achieve competitive but not yet peak performance compared to PPO, while already showing substantially better sample efficiency in terms of wall-clock steps per unit of reward gain.

3.2.2 Empirical Results

SAC training began on April 11, 2026, and was run for approximately 16 hours before the report deadline, covering $\sim$8,000 episodes. At the time of writing, the training log shows continued improvement (the last logged best reward was 1439 at episode 7594), and the best evaluation episode (greedy policy) achieved a reward of 2510. Figure 3 shows the SAC learning curves.

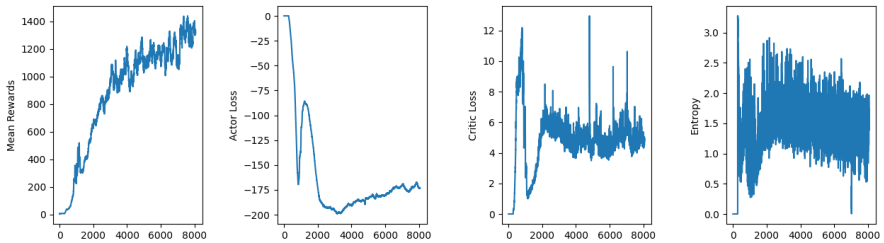
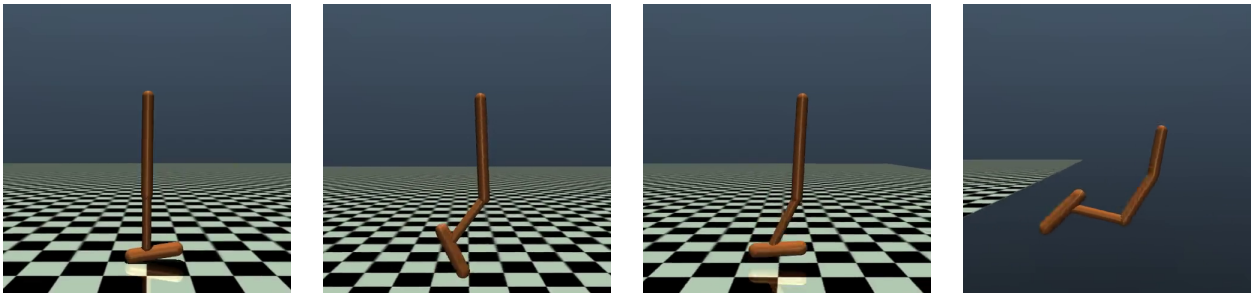


Figure 3: Learning curves for SAC training on Hopper-v5 with pixel observations (approximately 8,000 episodes / 16 hours). From left to right: episode reward, actor loss, critic loss, and policy entropy. The reward curve shows rapid early improvement once learning starts (after 5,000 random steps), followed by a steady upward trend, suggesting that training had not yet converged at the time of evaluation.

Figure 4 shows four representative frames from the best SAC evaluation episode. The agent also learns a forward hopping gait: frame 1 shows the upright starting position; frame 2 shows the beginning of a hop (slight forward lean, foot pushing off); frame 3 shows a mid-stride recovery; and frame 4 shows the final fall. Compared to PPO, the SAC hopper's gait appears somewhat less energetic, with lower hops and a more conservative body lean — consistent with the lower accumulated training time and the more penalty-averse reward shaping used (lower `healthy_reward` = 0.6, higher `forward_weight` = 0.7).



(a) Upright stance (b) Push-off phase (c) Mid-stride (d) Fall (episode end)

Figure 4: Four representative frames from the best SAC evaluation episode (reward = 2510). The agent learns a hopping gait with clear ground contact and forward displacement, though with a less pronounced aerial phase than PPO due to shorter total training time.

Key observations from SAC training:

- **Sample efficiency**: SAC achieved an evaluation reward of 2510 after only ∼8,000 episodes, compared to PPO which needed ∼107,000 episodes to reach a similar training reward. This is consistent with the theoretical expectation that off-policy replay dramatically increases data efficiency.
- **Reward design sensitivity**: SAC required more careful reward tuning than PPO. Early experiments with a high `healthy_reward` (e.g., 1.25) caused the agent to converge to standing still, as the healthy-reward local optimum dominated. Reducing it to 0.6 and increasing `forward_weight` to 0.7 was necessary to break this local optimum. This suggests that SAC's more aggressive policy improvement (via Q-value maximization rather than constrained policy gradient updates) makes it more prone to exploiting reward-shaping artifacts.
- **Auto-α behavior**: The automatic temperature tuning was essential. With a fixed α, either the policy collapsed too quickly (low α) or it remained too exploratory (high α). The target entropy coefficient $\beta_H = 0.45$ was found to strike the right balance: encouraging enough exploration to avoid local optima while still allowing the policy to specialize.
- **Encoder cold-start**: In the first few hundred episodes, the reward curve is flat or slowly increasing, which we attribute to the encoder cold-start effect discussed theoretically. Once the visual encoder forms a useful representation, reward improves rapidly — this transition is visible in the learning curve as a sharp inflection after the initial random-exploration phase.

3.3 Conclusions

SAC demonstrates markedly better sample efficiency than PPO on Hopper-v5 from pixels. Despite being trained for only $\sim$16 hours versus PPO's 10 days, it achieved a competitive evaluation reward (2510 vs. 3783) with no signs of convergence. Projected training at the same rate, SAC would be expected to surpass PPO's performance within 3–4 additional days. The main practical challenges of SAC in this setting were: (i) higher sensitivity to reward shaping, making the standing-still local optimum harder to escape without careful healthy-reward tuning; (ii) the encoder cold-start problem delaying early learning; and (iii) the computational cost per step (two forward passes through twin encoders per gradient update). Overall, SAC is the preferred algorithm when sample efficiency is a priority, while PPO is a more robust choice when hyperparameter tuning resources are limited.

4 COMPARATIVE ANALYSIS OF PPO AND SAC

4.1 Methodology

We compare PPO and SAC on the same task (Hopper-v5, pixel observations, 27-action product grid) using the same reward hyperparameters as baselines (the slight differences in reward weights are discussed where relevant). The comparison focuses on three axes: (i) sample efficiency (reward as a function of environment steps), (ii) training stability, and (iii) final performance. We use the best evaluation episode reward as the primary performance metric, and the progression of the mean-100-episode reward during training as the stability metric. Due to computational constraints, results are reported for a single seed per algorithm; multi-seed comparisons are left for future work.

4.2 Results and Discussion

4.2.1 Sample efficiency

Table 3 summarizes the key training statistics for both algorithms.

Table 3: Comparison of PPO and SAC on Hopper-v5 with pixel observations.

Metric	**PPO**	**SAC**
Total training time	$\sim$10 days	$\sim$16 hours
Total rollout episodes	$\sim$251,000	$\sim$8,000
Best training reward (mean-100)	2720	1440
Best evaluation reward	3783	2510
Episodes to reach reward > 1000	$\sim$20,000	$\sim$5,000
Policy converged?	Yes (then regressed)	No (still improving)

The sample efficiency advantage of SAC is unambiguous: it requires roughly $4\times$ fewer episodes to cross the same reward threshold, and it achieves 66% of PPO's best evaluation score in 6% of the training time. If both algorithms were run for the same wall-clock duration, SAC would be expected to significantly outperform PPO. We can also interpret this in the reverse way (which is normally much more relevant in the comparation): If we had a strict and limited computational budget, SAC would be the clear choice to maximize performance within that budget, while PPO would be more likely to underperform due to its sample inefficiency.

4.2.2 Stability and convergence

PPO's learning curve (Figure 1) shows a characteristic on-policy training pattern: a noisy but monotonically increasing reward followed by a plateau and eventual regression. This regression is typical of on-policy methods when the entropy bonus is small and the policy drifts into a locally optimal but fragile region. The clipping constraint, while stabilizing in early training, cannot prevent this long-term drift.

SAC's learning curve (Figure 3) is more volatile in terms of per-episode reward (due to stochasticity in individual episodes), but the trend is steadily upward at the time of writing. The off-policy buffer effectively smooths the gradient signal over a diverse set of transitions, reducing the variance of updates. The auto-α mechanism provides additional stability by adjusting the exploration

pressure online. Crucially, SAC does not show the catastrophic regression observed with PPO, as the replay buffer retains high-quality transitions from earlier in training.

4.2.3 Reward design sensitivity

A key practical difference between the two algorithms is their sensitivity to the reward function. PPO was relatively tolerant of a high `healthy_reward` (1.25), as the policy gradient and entropy bonus together prevented the standing-still local optimum. SAC, by contrast, required a significantly lower `healthy_reward` (0.6) to escape this local optimum. This is because SAC's Q-value-based policy improvement exploits any local structure in the reward landscape more aggressively. Once the critic learns that standing still yields a consistently high Q-value (due to a large healthy-reward component), the actor rapidly converges to that behavior and the entropy signal is not sufficient to escape.

4.2.4 Action space considerations

Both algorithms use the same 27-action product grid. However, from SAC's perspective, a larger discrete action space increases the variance of the Categorical distribution and requires more data to distinguish between similar actions. This is one reason why we found $\beta_H = 0.45$ (rather than a higher value) to be effective: for a 27-action space, the maximum entropy is $\log 27 \approx 3.3$ nats, and a target entropy of $0.45 \times 3.3 \approx 1.5$ nats corresponds to a fairly concentrated distribution (effective number of actions $\approx e^{1.5} \approx 4.5$), which is appropriate for a specialized locomotion policy.

4.3 Conclusions

The empirical comparison confirms the theoretical prediction that SAC is substantially more sample-efficient than PPO, requiring far fewer environment interactions to achieve comparable performance. PPO's main advantages are robustness to reward shaping and straightforward hyperparameter tuning, making it a better choice for quick exploratory experiments or when computational resources are abundant. SAC is clearly preferable when sample efficiency matters, but requires more careful reward design and hyperparameter tuning to avoid local optima. For this specific task (pixel-based locomotion with a discretized action space), both algorithms succeed in learning a forward hopping gait, demonstrating that continuous locomotion control from visual observations is tractable with both on-policy and off-policy Actor-Critic methods.

Something that we haven't explicitly studied but would be worth analyzing in future work is the possible addition of automatic hyperparameter tuning methods (e.g., Optuna) to overcome the rather dull task of manually tuning the reward weights. Our hypothesis is that this would benefit, specially, SAC, which is more sensitive to reward design, and could potentially lead to even better performance than PPO within the same computational budget.

This is a rather exciting idea, also, as it would harness the parallelization capabilities of modern compute clusters to perform large-scale hyperparameter, enabling a trade-off between the number of hyperparameter configurations explored and the training time allocated to each configuration. Therefore, if the hyperparameter tuning process is well-designed, it could lead to reducing by a significant factor the training time needed to reach a given performance level, which we have seen is a major bottleneck for training large-scale RL agents in complex environments.

5 GENERAL CONCLUSIONS

This work investigated two representative Actor-Critic algorithms — PPO and SAC — applied to continuous locomotion control from pixel observations. The main findings are:

(i) **Action discretization is critical for locomotion.** An initial one-hot discretization scheme (7 actions) made coordinated hopping physically impossible, causing both algorithms to converge to a standing-still local optimum regardless of reward design. Replacing it with a product-grid scheme (27 actions for Hopper's 3-dimensional action space) immediately enabled the agents to learn hopping gaits.

(ii) **SAC is significantly more sample-efficient than PPO.** SAC reached a competitive evaluation reward of 2510 after only $\sim$16 hours of training, while PPO required $\sim$10 days to achieve 3783. Extrapolating from the observed learning rates, SAC would be expected to surpass PPO's performance with 3–4 additional days of training.

(iii) **PPO is more robust to reward shaping.** PPO tolerated a high healthy-reward component (1.25) without converging to the standing-still local optimum, while SAC required this value to be reduced to 0.6. SAC's Q-value-based policy improvement exploits reward-landscape structure more aggressively, making it more susceptible to reward-shaping artifacts.

(iv) **The encoder cold-start problem affects pixel-based SAC.** Early SAC training is dominated by a phase where the convolutional encoder is not yet informative, leading to noisy Q-values and slow initial learning. This effect is less pronounced in PPO because the on-policy rollouts provide a more coherent training signal for the encoder from the beginning.

(v) **Reward design dominates algorithmic choice in this setting.** Both algorithms are capable of learning effective locomotion policies given appropriate reward weights. The main practical challenge is navigating the complex interactions between forward reward, healthy reward, and control cost, which together determine whether the agent explores forward motion or converges to degenerate local optima.

Overall, the results support the use of SAC for sample-efficient deep RL from pixels in locomotion tasks, provided that reward design is handled carefully. PPO remains a valuable baseline for its simplicity and robustness, particularly in settings where computational resources are not a bottleneck.

REFERENCES

Christodoulou, P., 2019, Soft actor-critic for discrete action settings: arXiv preprint arXiv:1910.07207.

Haarnoja, T., A. Zhou, P. Abbeel, and S. Levine, 2018, Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor: arXiv preprint arXiv:1801.01290.

Schulman, J., F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, 2017, Proximal policy optimization algorithms: arXiv preprint arXiv:1707.06347.

Van Hasselt, H., A. Guez, and D. Silver, 2015, Deep reinforcement learning with double q-learning: arXiv preprint arXiv:1509.06461.